03_clustering — k-means and hierarchical

Anish

2026-05-01

1. Goal

Fit ISLR-Ch.12 clusterers (k-means and hierarchical) on the PC scores from `02_PCA.Rmd`. Pick a `k` defensible under multiple criteria, validate stability via bootstrap, and save per-customer labels for `04_profiles.Rmd`.

2. Load and validate

```
pc       <- read_csv("../data/processed/customer_pc_scores.csv", show_col_types = FALSE)
customer <- read_csv("../data/processed/customer_features.csv",  show_col_types = FALSE)

stopifnot(
  "CustomerID" %in% names(pc),
  "CustomerID" %in% names(customer),
  !anyDuplicated(pc$CustomerID),
  setequal(pc$CustomerID, customer$CustomerID)
)

dim(pc)
```

```
## [1] 4334    4
```

```
glimpse(pc)
```

```
## Rows: 4,334
## Columns: 4
## $ CustomerID <dbl> 12346, 12347, 12348, 12349, 12350, 12352, 12353, 12354, 123~
## $ PC1        <dbl> -2.82716427, 3.34474112, 0.86447698, 0.42554416, -1.8673585~
## $ PC2        <dbl> -5.49318434, 0.22375290, -0.21402402, -2.26806776, -0.95978~
## $ PC3        <dbl> -0.37056405, -0.08693365, 0.79954652, -0.55067776, 0.420579~
```

```
Xpc <- pc |> select(starts_with("PC")) |> as.matrix()
rownames(Xpc) <- pc$CustomerID
n_pc <- ncol(Xpc)

sprintf("Clustering on %d PCs across %d customers", n_pc, nrow(Xpc))
```

```
## [1] "Clustering on 3 PCs across 4334 customers"
```

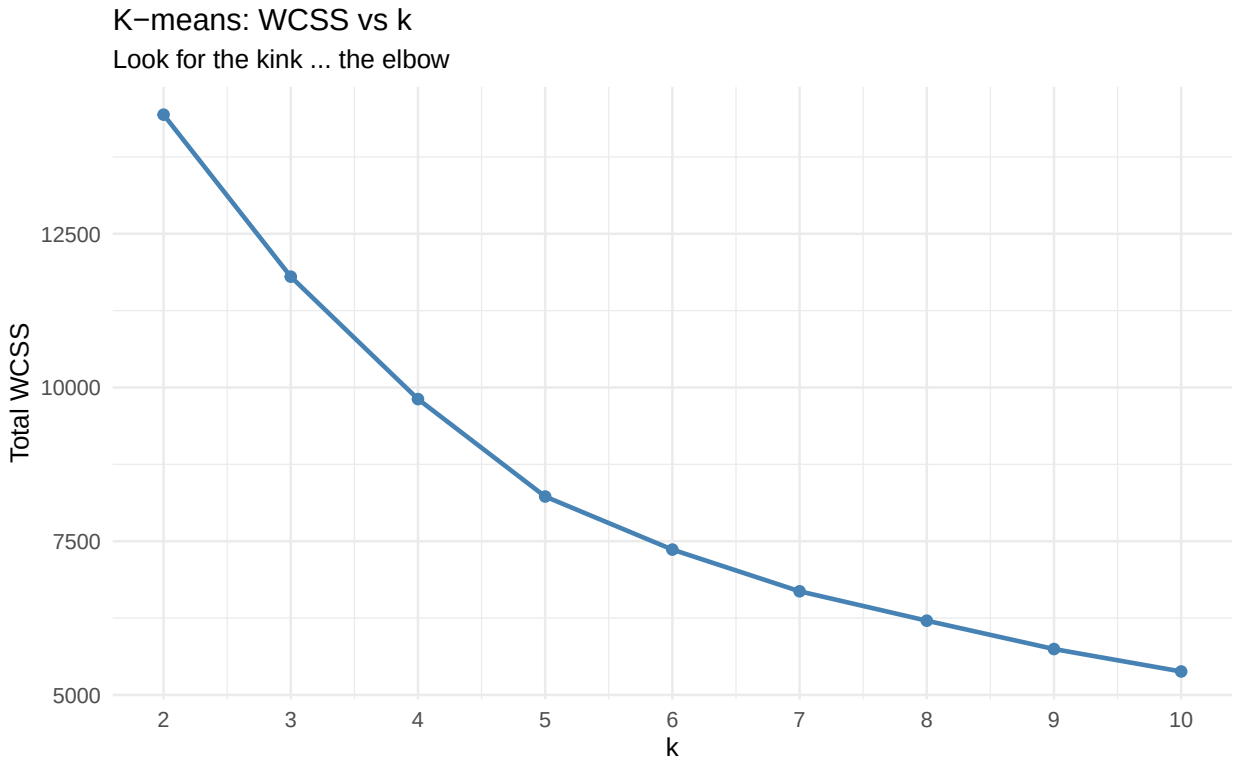
3. K-means — choosing k

3a. Elbow (within-cluster SSE)

The elbow plot is a heuristic: WCSS strictly decreases as `k` grows, because more clusters can fit the data more tightly. We look for the "kink" where adding another cluster stops paying off.

```
set.seed(1)
k_grid <- 2:10
wcss <- map_dbl(k_grid,
                \(k) kmeans(Xpc, centers = k, nstart = 25,
                            iter.max = 50)$tot.withinss)
elbow_df <- tibble(k = k_grid, wcss = wcss)

ggplot(elbow_df, aes(k, wcss)) +
  geom_line(colour = "steelblue", linewidth = 1) +
  geom_point(colour = "steelblue", size = 2) +
  scale_x_continuous(breaks = k_grid) +
  labs(title = "K-means: WCSS vs k",
       subtitle = "Look for the kink - the elbow",
       x = "k", y = "Total WCSS")
```

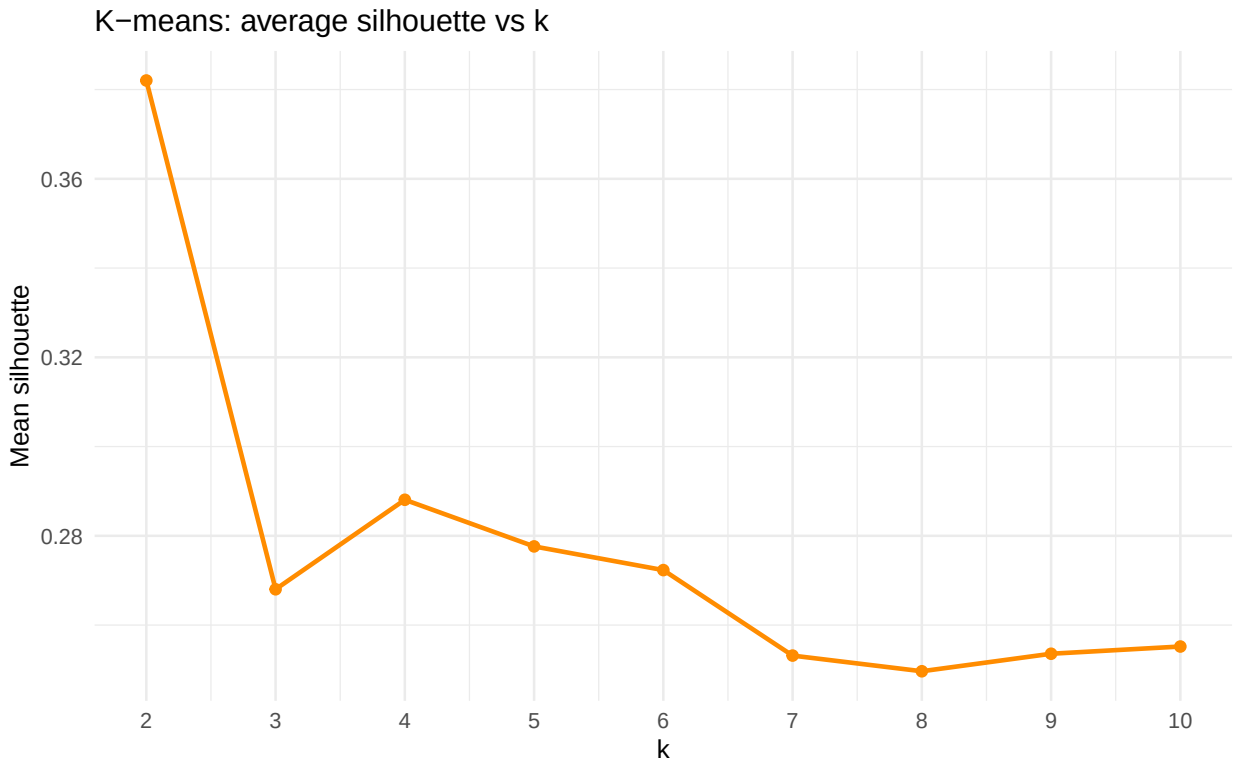


3b. Silhouette width

For each customer, silhouette width compares its average distance to others *in its own cluster* against its average distance to the *next-nearest cluster*. Mean silhouette ~ 0 means clusters overlap; ~ 1 means they're well separated. ISLR §12.4 doesn't formalise this, but it's the standard add-on.

```r
set.seed(1)
d <- dist(Xpc)
sil <- map_dbl(k_grid, \(k) {
  cl <- kmeans(Xpc, centers = k, nstart = 25, iter.max = 50)$cluster
  mean(silhouette(cl, d)[, "sil_width"])
})
sil_df <- tibble(k = k_grid, silhouette = sil)

ggplot(sil_df, aes(k, silhouette)) +
  geom_line(colour = "darkorange", linewidth = 1) +
  geom_point(colour = "darkorange", size = 2) +
  scale_x_continuous(breaks = k_grid) +
  labs(title = "K-means: average silhouette vs k",
       x = "k", y = "Mean silhouette")
```



3c. Gap statistic — Tibshirani (2001)

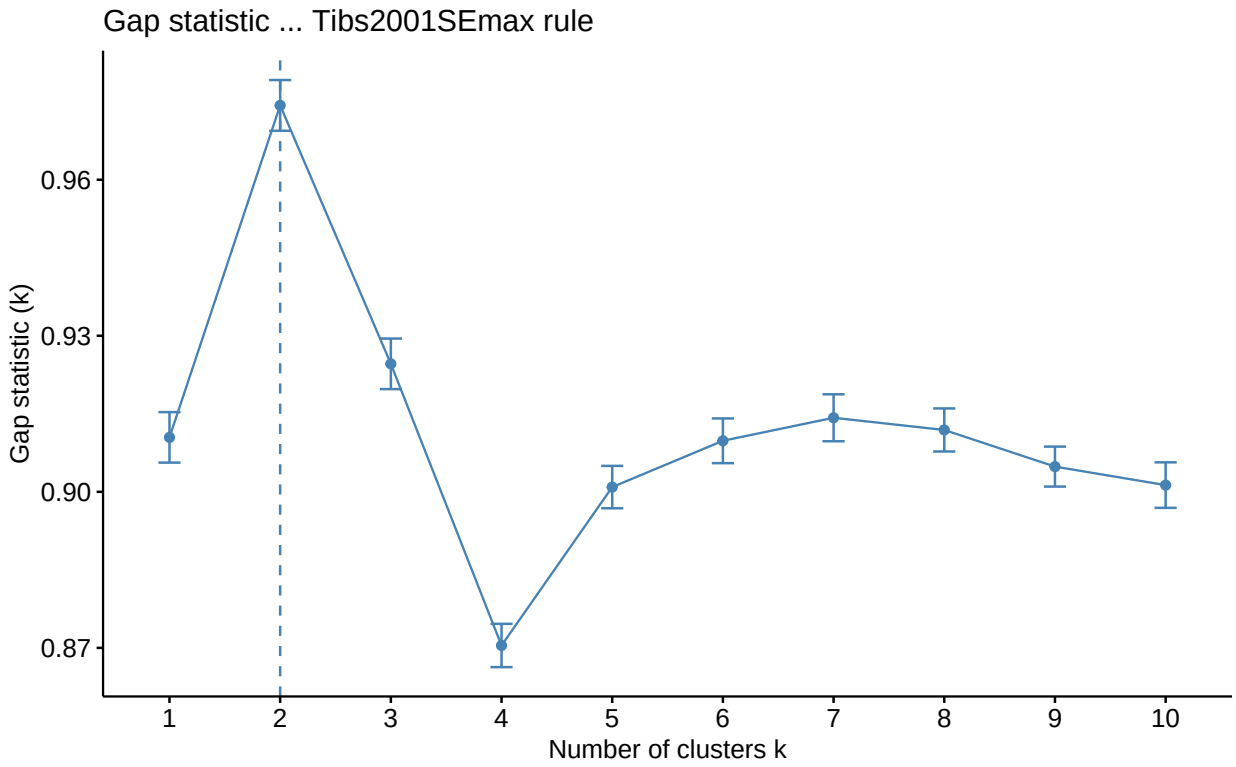
Compares the within-cluster dispersion in the actual data against the within-cluster dispersion under a uniform-noise reference distribution. The **SEmax rule** picks the smallest `k` such that no larger `k` is more than 1 standard error better — biased toward parsimony, which is exactly what we want for an interpretable customer segmentation.

```r
set.seed(1)
gap <- clusGap(Xpc, FUN = kmeans, nstart = 25, K.max = 10, B = 50)
print(gap, method = "Tibs2001SEmax")
```

```
## Clustering Gap statistic ["clusGap"] from call:
```

```
## clusGap(x = Xpc, FUNcluster = kmeans, K.max = 10, B = 50, nstart = 25)
## B=50 simulated reference sets, k = 1..10; spaceH0="scaledPCA"
##  --> Number of clusters (method 'Tibs2001SEmax', SE.factor=1): 2
##           logW   E.logW       gap      SE.sim
##  [1,] 8.099945 9.010415 0.9104701 0.004852826
##  [2,] 7.813058 8.787349 0.9742910 0.004876152
##  [3,] 7.704788 8.629396 0.9246082 0.004859627
##  [4,] 7.623985 8.494445 0.8704592 0.004158341
##  [5,] 7.535752 8.436666 0.9009134 0.004073224
##  [6,] 7.475036 8.384847 0.9098116 0.004298778
##  [7,] 7.425774 8.340011 0.9142366 0.004515049
##  [8,] 7.384932 8.296827 0.9118948 0.004139621
##  [9,] 7.350180 8.255031 0.9048513 0.003840894
## [10,] 7.315821 8.217114 0.9012931 0.004376231
```

```
fviz_gap_stat(gap) +
  labs(title = "Gap statistic - Tibs2001SEmax rule")
```



3d. Decision: k = 3 (with k = 2 as sensitivity)

Diagnostics summary:

- Silhouette peaks at `k = 2` and is competitive at `k = 3`.
- Gap statistic with the SEmax rule selects `k = 3`.
- Elbow is gradual; no decisive kink.

We choose **`k = 3`** because:

1. it is statistically defensible under SEmax (the most rigorous of the three diagnostics);
2. the silhouette difference between `k = 2` and `k = 3` is small enough that gap-statistic-driven parsimony wins;
3. `k = 3` produces interpretable high / mid / low value segments rather than the degenerate "above-average vs. below-average" two-way split that `k = 2` produces;
4. the persona-level profiles in `04_profiles.Rmd` are demonstrably more actionable at `k = 3`.

We carry **`k = 2`** as a sensitivity / simplified segmentation. The earlier "substantive prior of 4–6 clusters" framing is *not* what the diagnostics support.

```
set.seed(1)
k_star <- 3
km_fit <- kmeans(Xpc, centers = k_star, nstart = 50, iter.max = 100)

km_fit$size
```

```
## [1] 1804 1396 1134
```

```
round(km_fit$centers, 2)
```

```
##     PC1   PC2   PC3
## 1 -0.38 -0.59  0.20
## 2  2.22  0.31 -0.07
## 3 -2.14  0.56 -0.24
```

```
mean(silhouette(km_fit$cluster, d)[, "sil_width"])
```

```
## [1] 0.2692661
```

```
set.seed(1)
km_fit_k2 <- kmeans(Xpc, centers = 2, nstart = 50, iter.max = 100)
km_fit_k2$size
```

```
## [1] 2474 1860
```

```
mean(silhouette(km_fit_k2$cluster, d)[, "sil_width"])
```

```
## [1] 0.3810214
```

4. Sensitivity — scaling the PC scores

K-means on raw PC scores weights PC1 more heavily because PC1 has the largest variance by construction. Scaling each PC to unit variance gives every PC equal say in the distance metric. We check whether the segmentation is robust to this choice using the **adjusted Rand index** (`mclust::adjustedRandIndex`): aRI = 1 means identical labels (up to relabelling); aRI = 0 means agreement no better than chance.

```
set.seed(1)
km_raw    <- kmeans(Xpc,        centers = k_star, nstart = 50, iter.max = 100)
km_scaled <- kmeans(scale(Xpc), centers = k_star, nstart = 50, iter.max = 100)

aRI_scale <- adjustedRandIndex(km_raw$cluster, km_scaled$cluster)
list(adjusted_rand_raw_vs_scaled = round(aRI_scale, 3))
```

```
## $adjusted_rand_raw_vs_scaled
## [1] 0.327
```

If aRI $>$ ~0.7 the segmentation is essentially unchanged by scaling. If materially lower, the report should note that PC1 dominance is doing the heavy lifting and the conclusions are conditional on that choice.

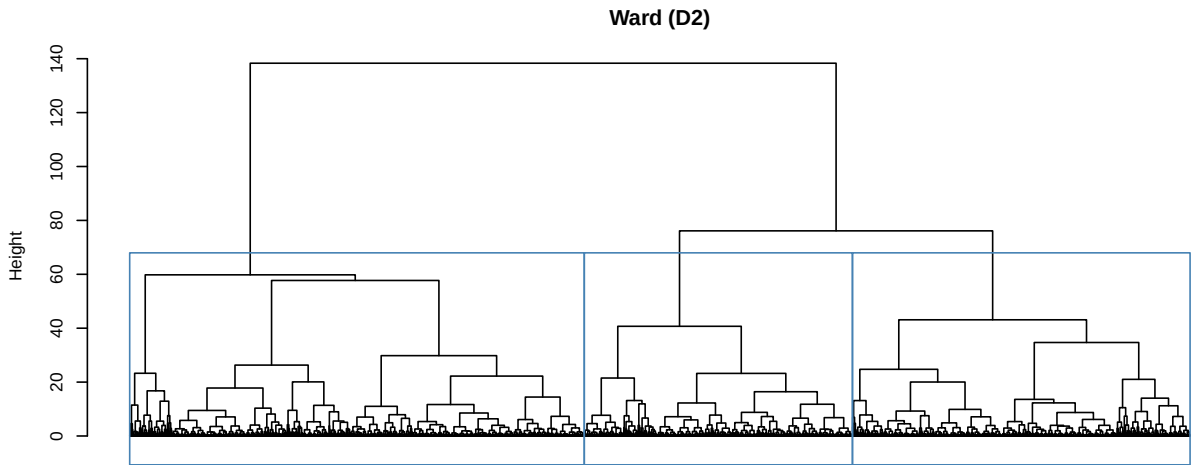
5. Hierarchical clustering

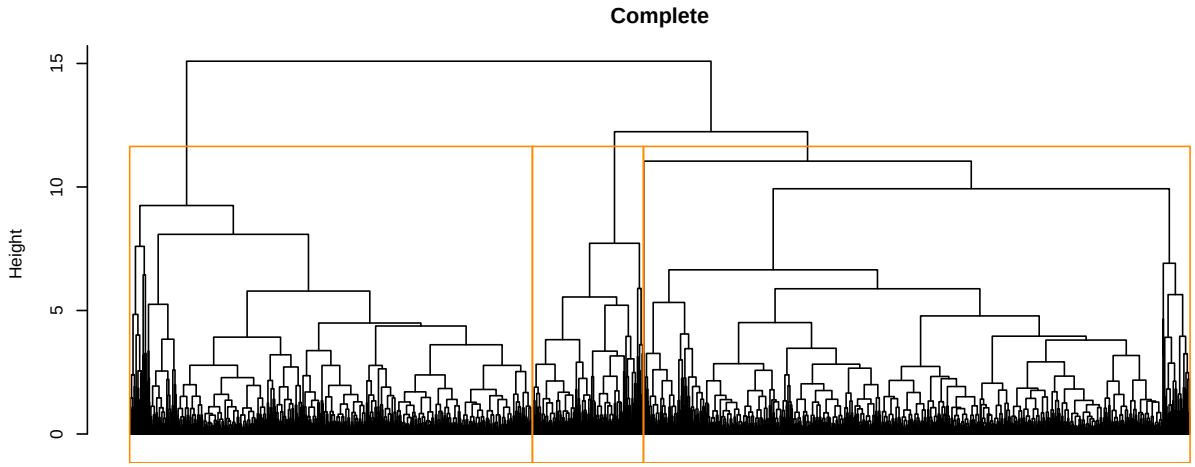
ISLR §12.4.2. Three linkages worth comparing:

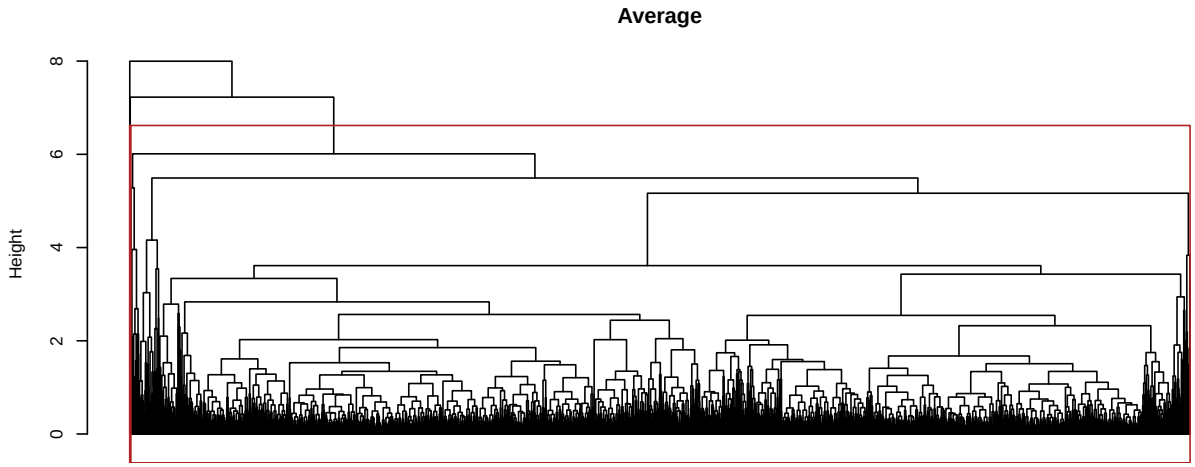
- **Ward (`ward.D2`)** — minimises within-cluster variance, philosophically aligned with k-means.
- **Complete** — ISLR's lab default; compact, similar-diameter clusters.
- **Average** — friendlier to elongated / unequal-density clusters; here as a robustness check.

```
hc_ward     <- hclust(d, method = "ward.D2")
hc_complete <- hclust(d, method = "complete")
hc_average  <- hclust(d, method = "average")
```

```
par(mfrow = c(3, 1), mar = c(2, 4, 3, 1))
plot(hc_ward,     labels = FALSE, hang = -1, main = "Ward (D2)",
     xlab = "", sub = "")
rect.hclust(hc_ward,     k = k_star, border = "steelblue")
plot(hc_complete, labels = FALSE, hang = -1, main = "Complete",
     xlab = "", sub = "")
rect.hclust(hc_complete, k = k_star, border = "darkorange")
plot(hc_average,  labels = FALSE, hang = -1, main = "Average",
     xlab = "", sub = "")
rect.hclust(hc_average,  k = k_star, border = "firebrick")
```








```
par(mfrow = c(1, 1))
```

```r
hc_cluster <- cutree(hc_ward, k = k_star)

cross_tab <- table(kmeans = km_fit$cluster, ward = hc_cluster)
cross_tab
```

```
##       ward
## kmeans    1    2    3
##      1  934    9  861
##      2  307 1087    2
##      3  139    0  995
```

```r
aRI_km_hc <- adjustedRandIndex(km_fit$cluster, hc_cluster)
list(adjusted_rand_kmeans_vs_ward = round(aRI_km_hc, 3))
```

```
## $adjusted_rand_kmeans_vs_ward
## [1] 0.383
```

```r
mean(silhouette(hc_cluster, d)[, "sil_width"])
```

```
## [1] 0.2275873
```

Interpretation — be honest, not optimistic. The cross-tab between k-means and Ward is *not* cleanly block-diagonal at `k = 3`; substantial off-diagonal mass means the algorithms disagree about boundaries. This is expected: k-means prefers spherical, equal-size clusters and Ward does not. The adjusted Rand index above quantifies